
Práctica 1: Shaders y materiales con Unity

Fecha de entrega: 27 de enero de 2026, 15:40

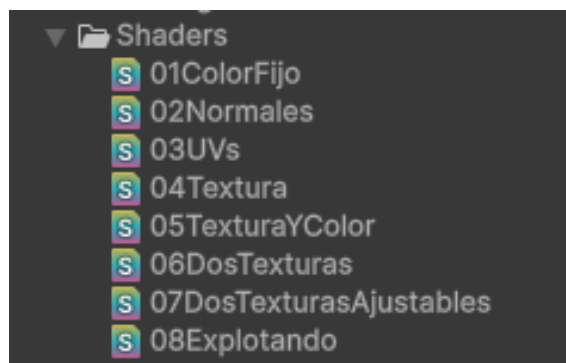
Objetivo: Repaso de IG con Unity. Primeros shaders y materiales

1. Instrucciones

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo shaders y materiales que se aplican sobre un cubo.

La entrega debe incluir únicamente los shaders individuales con los nombres *de ficheros* concretos indicados en el texto y una captura de la pestaña *Game* del editor de Unity en la que se vea el cubo con el material aplicado. Es importante que los nombres de los ficheros entregados coincidan con los pedidos. En el enunciado también se especifica qué nombre deben tener las variables HLSL de los parámetros del material.

Si vas creando los shaders en la carpeta `Shaders` del proyecto, al final de la práctica deberías ver algo así:



En la entrega habrá un fichero con extensión `.shader` por cada uno de los archivos que aparecen en la imagen, así como una captura del resultado de aplicar ese material con

extensión `.png` o `.jpg`. Por ejemplo, se espera un fichero `01ColorFijo.shader` y un `01ColorFijo.png` (o `.jpg`).

Para el desarrollo se recomienda crear la escena base con el cubo y duplicarla de forma que en cada escena el cubo utilice un material y shader distinto.

2. Punto de partida

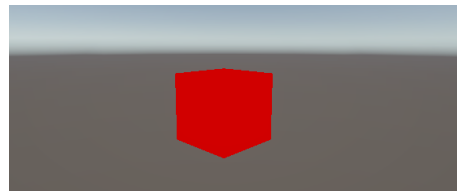
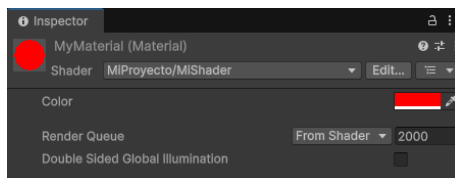
Crea un proyecto 3D en Unity que utilice el *Universal Render Pipeline* con una escena con:

- El contenido por defecto de la plantilla de Unity, con skybox, cámara y luz direccional.
- La cámara con la localización (*L*), escalado (*S*) y rotación (*R*) siguientes: *L*: (-3, 1, -3), *S*: (1, 1, 1), *R*: (10, 45, 0) .
- Un cubo en *L*: (0, 0, 0), *S*: (1, 1, 1), *R*: (0, 0, 0) .

3. Color fijo

Crea un (*Unlit shader*) que tenga un color fijo configurable desde el material. El parámetro del color en el código HLSL debe llamarse `_Color` y cuyo valor por defecto sea el verde.

Utilízalo en el cubo con un material que establezca el color rojo.



4. Pintando las normales

Crea otro shader y material llamado `02Normales`. En el VS se determina el color de cada vértice haciéndolo igual al valor absoluto de su normal (la normal viene en la geometría con un `float3` de semántica `NORMAL`). El color (un `float4` de semántica `COLOR`) es pasado al PS.

La normal no se transforma antes de interpretarla como color, por lo que se utiliza la que venga en la geometría independientemente del lugar y rotación que ocupe el modelo en el mundo.

Antes de hacerlo, ¿qué resultado esperas obtener al aplicarlo en un cubo? ¿Y en una esfera? Si al ver la imagen final no es lo que esperabas, asegurate de que entiendes por qué aparece lo que aparece.

5. Pintando las coordenadas de textura

Crea otro shader y material llamado `03UVs`. En el VS se determina el color de cada vértice haciendo que la componente roja y verde coincidan con las coordenadas de textura

u y v respectivamente y con componente azul nula.

Recuerda que las coordenadas de textura vienen en la geometría con un `float2` de semántica `TEXCOORD0`.

Antes de hacerlo, ¿qué resultado esperas obtener al aplicarlo en un cubo? Si al ver la imagen final no es lo que esperabas, asegurate de que entiendes por qué aparece lo que aparece.

6. Textura

Crea otro shader y material llamado `04Textura` que utilice una textura pasada como parámetro al material. La forma de definir la propiedad en Shader Lab es:

```
Properties
{
    _MainTex ("Base (RGB)", 2D) = "" {}
}
```

El tipo en HLSL es `sampler2D` y la forma de muestrear la textura es utilizando la función `tex2D`, que recibe el `sample` y las coordenadas.

Utiliza la textura `BeachStones.jpg` proporcionada para probarlo.

7. Textura y color fijo

Extiende el shader y material anterior (con nombre `05TexturaYColor`) para que admita un parámetro de tipo color y lo mezcle con la textura. Haz la mezcla utilizando el producto de Hadamard (multiplicación de cada componente).

Utilízalo con la textura `checker.jpg` proporcionada. ¿Qué esperas ver si se combina con el color rojo, (1, 0, 0)?

8. Dos texturas

Crea un nuevo shader (con nombre `07DosTexturas`) que tenga dos parámetros de tipo textura que se mezclen de igual forma que en el material anterior.

Las propiedades deben ser:

```
Properties
{
    _MainTex ("Base (RGB)", 2D) = "" {}
    _SecondaryTex ("Secondary (RGB)", 2D) = "" {}
}
```

Pruébalo con las texturas `BeachStones.jpg` y `checker.jpg`.

9. Dos texturas ajustables

Crea un nuevo shader `07DosTexturasAjustables` que reciba también dos texturas principales. En lugar de mezclar sus colores, el color final de cada pixel será la media ponderada del color de ambas texturas. El factor de ponderación será *el alpha* de una tercera textura. De esta forma, cuando el alpha sea 0 el color final coincidirá con el de una de las dos texturas principales y si es 1 coincidirá con el de la otra.

Las propiedades deben ser:

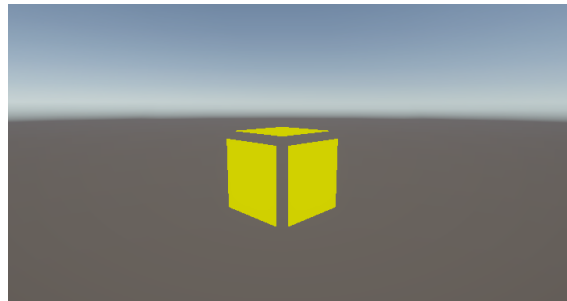
Properties

```
{
    _MainTex ("Base (RGB)", 2D) = "" {}
    _SecondaryTex ("Secondary (RGB)", 2D) = "" {}
    _BlendTex ("Alpha Blended (___A)", 2D) = "" {}
}
```

Para probarlo utiliza, además de las dos texturas anteriores, la `DegradadoGris.es.png` (tendrás que ajustar las propiedades de exportación de Unity para conseguir que tenga alpha).

10. Explotando

Crea un nuevo shader `08Explotando` utilizando el primero como base (`01ColorFijo`) que tenga un resultado como el siguiente:



El shader “separa” las caras un factor que se da como parámetro real:

```
Properties
{
    _Color("Color", Color) = (0, 1, 0)
    _Scale("Scale", float) = 0.10
}
```

La separación debe fluctuar con el tiempo entre 0 y ese valor. Hay distintas formas de acceder al tiempo. Una de ellas es utilizando la variable `_Time`, un `float4` que en cada componente tiene los valores `tiempo/20`, `tiempo`, `2*tiempo` y `3*tiempo`. Se puede utilizar también `_SinTime` que en la última componente tiene el seno del tiempo.

11. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde `NN` representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- Los ficheros de shaders y capturas indicados anteriormente con los nombres exactos.
- Incluir además un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.